

Wksh 2_1: Functions and Procedures

Examples:

Pseudo code example	Equivalent in C language (with some C++ facilities)												
Algo Exmpl Var X, Y, Z, W : Integer Function Fact(N : Integer) : Integer Var F, i : Integer Begin F ← 1 For i ← 2 to N do F ← F * i EndFor Return(F) End Begin Read (X, Y) Z ← Fact(X) + 1 W ← Fact(Y) * 2 Write(Z + W) End	<pre>#include <iostream> using namespace std; int X, Y, Z, W; int Fact(int N) { int F, i; F = 1; for(i=2;i<=N;i++) { F = F * i; } return(F); } int main() { cin >> X >> Y; Z = Fact(X)+1; W = Fact(Y)*2; cout << Z + W; }</pre> <table><tr><td>X</td><td>Y</td><td>Screen</td></tr><tr><td>5</td><td>3</td><td>133</td></tr></table>	X	Y	Screen	5	3	133						
X	Y	Screen											
5	3	133											
Algo Exmpl Var X, Y : Real Procedure Permut(Ref A, B : Real) Var t : Real Begin t ← A A ← B B ← t End Begin Read (X, Y) Permut(X, Y) Write(X, " ", Y) End	<pre>#include <iostream> using namespace std; float X, Y; void Permut(float &A, float &B) { float t; t = A; A = B; B = t; } int main() { cin >> X >> Y; Permut(X, Y); cout << X << " " << Y; }</pre> With &: <table><tr><td>X</td><td>Y</td><td>Screen</td></tr><tr><td>4</td><td>13</td><td>13 4</td></tr></table> Without &: <table><tr><td>X</td><td>Y</td><td>Screen</td></tr><tr><td>4</td><td>13</td><td>4 13</td></tr></table>	X	Y	Screen	4	13	13 4	X	Y	Screen	4	13	4 13
X	Y	Screen											
4	13	13 4											
X	Y	Screen											
4	13	4 13											

Exercise 1.01:

Algo Exo1_1_a

Var

X, Y, Z : Integer

Function Fctn(N : Integer) : Integer

Begin

Return(N * N * N)

End

Begin

Read(X, Y)

Z ← Fctn(X) - Fctn(Y)

Write(Z)

End

X	Y	Sreen
3	2	19

Algo Exo1_1_b

Var

X, Y, Z : Integer

Function Fn(B, P : Integer) : Integer

Var

Res, i: Integer

Begin

Res ← 1

For i ← 1 **to** P **do**

Res ← Res * B

EndFor

Return(Res)

End

Begin

Read(X,Y)

Z ← Fn(X,Y) + Fn(Y,X)

Write(Z)

End

X	Y	Sreen
3	2	17

Exercise 1.11:

Algo Exo1_11

Var

X, Y, Z, W : Integer

Procedure Prc(A: Integer, **Ref** B : Integer)

Begin

If A < 0 **then**

B ← B - 1

Else

While A ≠ 0 **do**

B ← B * 2

A ← A - 1

EndWhile

EndIf

End

Begin

Read(X, Y)

Prc(X, Y)

Write(X, Y)

Z ← X + Y

W ← Y

Prc(Z, W)

Write(Z, W)

End

Pour X = -4 and Y = 2 :

X = Y =

Z = W =

Pour X = 0 and Y = 2 :

X = Y =

Z = W =

Exercise 1.03_c (Home work):

Algo Exo1_3_c

Var

x, Form : Real

i : Integer

Function Fact(N : Integer) : Integer

Var

F, i: Integer

Begin

F ← 1

For i ← 2 **to** N **do**

F ← F * i

EndFor

Return(F)

End

Function Power(B : Real, P : Integer) : Real

Var

Res : Real

i: Integer

Begin

If P < 0 **then**

P ← -P

B ← 1/B

EndIf

Res ← 1

For i ← 1 **to** P **do**

Res ← Res * B

EndFor

Return(Res)

End

Begin

Read(x)

Form ← 1

For i ← 1 **to** 9 **do**

Form ← Form + Power(x,i) / Fact(i)

EndFor

Write(Form)

End

x	S
0	1
1	2.71828
2	7.38871
2.3	9.97275

Pseudo code	Equivalent in C++ language
V : Boolean V $\leftarrow$ True	bool V; V = true;

Exercise 1.05_a (Home work):

Algo Exo1_5_a

Var

Nb, X, CoP, SP : Integer

Function Prime(N : Integer) : Boolean

Var

Co, d : Integer

Begin

Co $\leftarrow$ 0

For d $\leftarrow$ 1 **to** N **do**

If N mod d = 0 **then**

 Co $\leftarrow$ Co + 1

EndIf

EndFor

If Co = 2 **then**

 Return(True)

Else

 Return(False)

EndIf

End

Nb	SP
1	2
3	10
50	5117
100	24133

Begin

Read(Nb)

CoP $\leftarrow$ 0

X $\leftarrow$ 2

SP $\leftarrow$ 0

While CoP < Nb **do**

If Prime(X) = True **then**

 Write(X)

 CoP $\leftarrow$ CoP + 1

 SP $\leftarrow$ SP + X

EndIf

 X $\leftarrow$ X + 1

EndWhiles

Write(SP)

End

Exercise (Home work):

Definition :

We will define a '*Jim*'¹ 'ج' number as being any strictly positive integer whose division by 100 equals the number of its divisors.

Example of *Jim* numbers:

The divisors of 211 are: 1 and 211. $\Rightarrow \text{nbr_div}(211) = 2 = 211 \text{ div } 100$

The divisors of 403 are: 1, 13, 31 and 403. $\Rightarrow \text{nbr_div}(403) = 4 = 403 \text{ div } 100$

The divisors of 1204 are: 1, 2, 4, 7, 14, 28, 43, 86, 172, 301, 602 and 1204. $\Rightarrow \text{nbr_div}(1204)=12=1204 \text{ div } 100$

Write a function `nbr_div` which has a strictly positive integer as parameter and returns the number of its divisors, in order to use it for:

1. Writing a program that checks if a number A is '*Jim*' or not.
2. Writing a program that displays all '*Jim*' numbers less than N.
3. Writing a program that displays the first Nb *Jim* numbers.

Question 1	
A	Screen
53	No
117	No
250	No
211	Yes
403	Yes
1204	Yes

Question 2	
N	Screen
250	211 223 227 229 233 239 241

Question 3	
Nb	Screen
20	211 223 227 229 233 239 241 251 257 263 269 271 277 281 283 293 361 403 407 411

¹ This name has been assigned for ease of understanding.

Exercise 1.02 (Additional):

Algo Exo1_2_b

Var

X : Integer

Function Fact(N : Integer) : Integer

Var

F, i: Integer

Begin

F ← 1

For i ← 2 **to** N **do**

F ← F * i

EndFor

Return(F)

End

X	Screen
3	6
5	120
6	720

Begin

Repeat

Read(X)

Until X ≥ 0

Write(Fact(X))

End

Algo Exo1_2_c

Var

Form1 : Real

i : Integer

Function Fact(N : Integer) : Integer

Var

F, i: Integer

Begin

F ← 1

For i ← 2 **to** N **do**

F ← F * i

EndFor

Return(F)

End

Begin

Form1 ← 1

For i ← 1 **to** 9 **do**

Form1 ← Form1 + 1/Fact(i)

/* in C language :

Form1 = Form1 + 1.0 /Fact(i) */

EndFor

Write(Form1)

End

Screen
2.718282

Algo Exo1_2_d

Var

Form2 : Real

i : Integer

Function Fact(N : Integer) : Integer

Var

F, i: Integer

Begin

F ← 1

For i ← 2 **to** N **do**

F ← F * i

EndFor

Return(F)

End

Begin

Form2 ← 1

For i ← 1 **to** 9 **do**

Form2 ← Form2 + (9-i+1)/Fact(i)

/* In C language :

Form2 = Form2 +

float(9-i+1)/Fact(i) */

EndFor

Write(Form2)

End

Screen
15.464563

Exercise 1.06 (Additional):

Algo Exo1_6_a

Var

i, N1, N2 : Integer

Function LCM(A, B : Integer) : Integer

Var

Ainit, Binit: Integer

Begin

Ainit $\leftarrow$ A

Binit $\leftarrow$ B

While A $\neq$ B **do**

If A > B **then**

B $\leftarrow$ B + Binit

Else

A $\leftarrow$ A + Ainit

EndIf

EndWhile

Return(A)

End

Begin

N1 $\leftarrow$ 1

For i $\leftarrow$ 1 **to** 4 **do**

Repeat

Read(N2)

Until N2 > 0

N1 $\leftarrow$ LCM(N1,N2)

EndFor

Write(N1)

End

Algo Exo1_6_b

Var

N1, N2, i : Integer

Procedure LCM2(A, B : Integer; **Ref** R : Integer)

Var

Ainit, Binit: Integer

Begin

Ainit $\leftarrow$ A

Binit $\leftarrow$ B

While A $\neq$ B **do**

If A > B **then**

B $\leftarrow$ B + Binit

Else

A $\leftarrow$ A + Ainit

EndIf

EndWhile

R $\leftarrow$ A

End

Begin

N1 $\leftarrow$ 1

For i $\leftarrow$ 1 **to** 4 **do**

Repeat

Read(N2)

Until N2 > 0

LCM2(N1,N2,N1)

EndFor

Write(N1)

End

The Least Common
Multiple of
2, 3, 6 and 10
Is **30**

Exercise 1.07 (Additional):

Function Binary(N : Integer) : Integer

Var

S, P : Integer

Begin

S $\leftarrow$ 0

P $\leftarrow$ 1

While N $\neq$ 0 **do**

S $\leftarrow$ S + (N mod 2) * P

N $\leftarrow$ N Div 2

P $\leftarrow$ P * 10

EndWhile

Return(S)

End

To be able to execute the function, it
should be called a main program.

Binary(2) = 10

Binary(5) = 101

Binary(23) = 10111

Exercise 1.09 (Additional):

Algo Exo1_9

Var

X, Y, X2, Y2 : Real

Procedure Reading()

Begin

Read(X,Y)

Repeat

Read(X2,Y2)

Until X2 ≠ X or Y2 ≠ Y

End

Procedure Center()

Var

XCtr, YCtr: Real

Begin

$XCtr \leftarrow (X+X2)/2$

$YCtr \leftarrow (Y+Y2)/2$

Write(" ", XCtre , " ", YCtre , " ")

End

Procedure Norm()

Var

Nrm: Real

Begin

$Nrm \leftarrow \sqrt{(X2-X)*(X2-X) + (Y2-Y)*(Y2-Y)}$

Write("The Norm is : ", Nrm)

End

Procedure Lnr_Eq()

Var

Alpha, Beta: Real

Begin

If X = X2 **then**

Write("X = ", X2)

Else

$Alpha \leftarrow (Y2-Y)/(X2-X)$

$Beta \leftarrow Y - Alpha * X$

Write("Y = ", Alpha, ".X + ", Beta)

EndIf

End

Begin

Reading()

Center()

Norm()

Lnr_Eq ()

End

X	Y	X2	Y2	Screen
2	3	4	3	(3 , 3) The Norm is : 2 $Y = 0.X + 3$
1	1	5	4	(3 , 2.5) The Norm is : 5 $Y = 0.75.X + 0.25$